**ИНСТИТУТ ТЕХНОЛОГИЙ УПРАВЛЕНИЯ**
Центр технологий 1С

Дисциплина: «Проектирование и архитектура программного обеспечения»

# ПРАКТИЧЕСКОЕ ЗАДАНИЕ

## «Выбор темы проекта. Сбор требований»

Тема проекта № 12:

**«Генератор отчётов на основе SQL-подобного языка»**

(рабочее название — SQLReport)

**Выполнили:** студенты группы ЭПБО-02-25
Батырбеков Ыйман Адилетович
Рабаданов Абудулманап Султанмутович

**Преподаватель:** Яковлев Михаил Сергеевич

**Семестр:** 1 семестр, 2026–2027 уч. г.

Москва, 2026

## СОДЕРЖАНИЕ

**1. Выбор темы проекта.....3**
1.1. Описание идеи.....3
1.2. Обоснование выбора.....3
**2. Заинтересованные стороны.....4**
**3. Методы сбора требований.....4**
3.1. Выбор методов.....4
3.2. Анализ документов и существующих решений.....5
3.3. Интервью.....6
3.4. User Stories и Story Mapping.....7
**4. Требования по иерархии Вигерса.....8**
4.1. Бизнес-требования.....8
4.2. Пользовательские требования.....9
4.3. Функциональные требования.....10
4.4. Нефункциональные требования.....12
**5. Потребности заказчика и детали реализации.....13**
**6. Трассировка требований.....14**
**Заключение.....15**

# 1. Выбор темы проекта

## 1.1. Описание идеи

Выбрана тема № 12 из перечня проектов — «Генератор отчётов на основе SQL-подобного языка» (SQLReport). Библиотека и консольная программа на языке Java, которая принимает текстовый запрос на SQL-подобном языке (например, SELECT name, avg(grade) FROM students.csv GROUP BY name), читает один или несколько CSV-файлов, выполняет над ними выборку, фильтрацию, соединение, группировку с агрегатными функциями, сортировку и выводит результат в виде отчёта — таблицы в консоли либо файла в формате CSV, HTML или Markdown.

Пример запроса, который должен выполнять продукт:

```
SELECT name, avg(grade) AS avg_grade, count(*) AS n
FROM students.csv
WHERE grade >= 3
GROUP BY name
ORDER BY avg_grade DESC
LIMIT 3;
```

## 1.2. Обоснование выбора

Тема выбрана по следующим причинам:

- CSV — самый распространённый формат выгрузки данных (из 1С, LMS, Excel, систем мониторинга), а потребность «быстро посчитать сводку» по нему возникает постоянно;
- проект целиком реализуется на Java без СУБД и покрывает ключевые темы курса: многоуровневую архитектуру, паттерны Interpreter, Facade, Iterator, Strategy, работу с файлами и обработку ошибок;
- объём работ хорошо масштабируется: минимальная версия (SELECT … FROM … WHERE) реализуема за несколько недель, а JOIN, HAVING, новые форматы вывода добавляются итеративно;
- результат легко проверить: ответ движка можно сверить с тем же запросом в SQLite или Excel.

Ограничения задания учтены с самого начала: проект реализуется на Java, а ключевые компоненты — чтение CSV, лексический и синтаксический

анализ запроса, выполнение и форматирование — пишутся самостоятельно, без использования готовых SQL-движков.

## 2. Заинтересованные стороны

Перед сбором требований определены заинтересованные стороны — источники требований и участники их согласования.

Таблица 1 — Заинтересованные стороны проекта

| **Заинтересованная сторона** | **Интерес / ожидания** | **Роль** | **Влияние** |
|---|---|---|---|
| Аналитик данных / методист | Получать сводные отчёты по выгрузкам CSV без Excel-формул и без установки СУБД | Основной пользователь CLI | Высокое |
| Разработчик-интегратор | Встроить генерацию отчётов в своё Java-приложение (бот, веб-сервис, cron-задачу) | Пользователь Java API | Среднее |
| Преподаватель (заказчик учебного проекта) | Проверить качество архитектуры, документации и кода; соответствие ограничениям (Java, без подмены ядра библиотеками) | Приёмка результата | Высокое |
| Команда разработки | Реализуемый объём, понятная модульная архитектура, тестируемость | Исполнитель | Среднее |

## 3. Методы сбора требований

### 3.1. Выбор методов

Для сбора требований использована комбинация традиционных и agile-ориентированных методов. Такой выбор обусловлен тем, что у продукта есть готовые аналоги (их анализ даёт «нижнюю планку» ожиданий), есть конкретные представители пользователей (их потребности выясняются в интервью), а разработка ведётся итеративно (story map позволяет разбить объём на релизы).

Таблица 2 — Использованные методы сбора требований

| **Метод** | **Что использовалось** | **Результат** |
|---|---|---|
| Анализ документов и существующих решений | RFC 4180, подмножество стандарта SQL, документация SQLite, csvkit, q, DuckDB; текст задания курса | Ограничения и обязательные возможности, понятийный аппарат, перечень ожидаемых конструкций языка |

| Метод | Что использовалось | Результат |
|---|---|---|
| Интервью | Полуструктурированное интервью с представителем заказчика и потенциальными пользователями (аналитик, разработчик) в рамках учебного сценария | Реальные сценарии использования, объёмы данных, требования к ошибкам и форматам вывода |
| User Stories | Формулирование потребностей от лица ролей с критериями приёмки | 12 пользовательских историй US-01…US-12 |
| Story Mapping | Раскладка историй по шагам пользовательского пути и по релизам | План релизов: MVP, релиз 2, релиз 3 |

### 3.2. Анализ документов и существующих решений

Проанализированы следующие источники:

- RFC 4180 «Common Format and MIME Type for CSV Files» — правила экранирования кавычек, разделителей и переводов строк внутри полей;
- подмножество стандарта SQL (ISO/IEC 9075) для оператора SELECT — порядок логического выполнения: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT;
- документация аналогов (SQLite, csvkit, q, DuckDB) — какие конструкции пользователи ожидают «по умолчанию» и какие сообщения об ошибках считаются удобными;
- текст задания по курсу (ограничения: язык Java, сторонние библиотеки не должны подменять реализацию проекта).

Сравнение существующих решений, которые сейчас используются для получения отчётов по CSV-файлам:

Таблица 3 — Анализ аналогов

| Решение | Достоинства | Недостатки для наших пользователей |
|---|---|---|
| MS Excel / LibreOffice Calc (сводные таблицы) | Привычный интерфейс, сводные таблицы, графики | Ручные действия, отчёт нельзя повторить одной командой; проблемы с большими файлами и кодировками CSV |
| SQLite (команда .import) | Полноценный SQL | Нужно сначала импортировать CSV в базу, все поля импортируются как текст, отдельный инструмент вне Java-приложения |

| Решение | Достоинства | Недостатки для наших пользователей |
|---|---|---|
| csvkit (csvsql), q | SQL-запросы прямо к CSV из командной строки | Требуют Python-окружения; внутри используют SQLite, т. е. не встраиваются в Java-код как библиотека |
| DuckDB | Быстрый аналитический SQL, читает CSV напрямую | Тяжёлая внешняя СУБД; для учебного проекта её использование подменило бы реализацию ядра |

**Вывод.** Нишу продукта образует сочетание трёх свойств, которого нет у аналогов одновременно: SQL-подобный язык, прямое чтение CSV без импорта и установки СУБД, возможность встроить движок в Java-приложение как библиотеку. Из анализа стандарта SQL взят логический порядок выполнения запроса, из RFC 4180 — правила разбора CSV.

### 3.3. Интервью

Проведено полуструктурированное интервью. В рамках учебного сценария роль заказчика выполняет преподаватель, а в роли пользователей выступают аналитик, работающий с выгрузками оценок и посещаемости, и Java-разработчик, которому нужна библиотека для автоматической рассылки отчётов. Ниже приведён сводный протокол интервью.

Таблица 4 — Протокол интервью

| № | Вопрос | Ответ (сводно) |
|---|---|---|
| 1 | Какие данные вы чаще всего обрабатываете и откуда они берутся? | Выгрузки из учётных систем и LMS в CSV: ведомости оценок, посещаемость, продажи. Файлы от сотен строк до ~1 млн строк, кодировка UTF-8, иногда Windows-1251, разделитель «;» или «,». |
| 2 | Какие отчёты нужны регулярно? | Средний балл по студенту/группе, количество записей по категориям, топ-N, сводки за период. Иногда нужно соединить два файла (например, оценки и список групп). |
| 3 | Как вы делаете это сейчас и что не устраивает? | Excel: сводные таблицы руками. Долго, легко ошибиться, при новой выгрузке всё повторяется заново. Файлы > 500 тыс. строк открываются медленно. |
| 4 | Знаком ли вам SQL? | На базовом уровне: SELECT, WHERE, GROUP BY, ORDER BY. Сложных подзапросов не пишем. |
| 5 | В каком виде нужен результат? | Посмотреть таблицу сразу в консоли; сохранить в CSV для дальнейшей работы; HTML или Markdown — чтобы вставить в письмо, вики или отчёт. |

| № | Вопрос | Ответ (сводно) |
|---|---|---|
| 6 | Что должно происходить при ошибке в запросе? | Понятное сообщение на русском с указанием, где ошибка (строка, позиция), и подсказкой; программа не должна «падать» со стек-трейсом. |
| 7 | Нужна ли интеграция с другими программами? | Да, со стороны разработчиков: вызвать движок из своего Java-кода (бот, ежедневная рассылка отчётов) и при необходимости добавить свою агрегатную функцию, например медиану. |

**Выводы по интервью:** основная потребность — быстрые повторяемые сводки с группировкой и агрегатами; критичны понятные сообщения об ошибках, поддержка разных разделителей и кодировок, несколько форматов вывода; для разработчиков — программный интерфейс и расширяемость функций. Сложные конструкции SQL (подзапросы, оконные функции) пользователям не нужны.

### 3.4. User Stories и Story Mapping

По результатам анализа и интервью потребности пользователей сформулированы в виде пользовательских историй (полный перечень — в разделе 4.2). Истории разложены на карте: по горизонтали — шаги пользовательского пути (backbone), по вертикали — релизы по приоритету. Верхняя строка образует минимально жизнеспособный продукт (MVP).

Таблица 5 — Карта пользовательских историй (Story Map)

| Релиз | Подготовить данные | Написать запрос | Выполнить запрос | Получить отчёт | Автоматизировать |
|---|---|---|---|---|---|
| MVP (релиз 1) | Указать CSV-файл с заголовком; автоопределение типов | SELECT, FROM, WHERE, ORDER BY, LIMIT; псевдонимы AS | Выполнить запрос в REPL, получить ошибку с позицией | Таблица в консоли | Запуск с параметром -q "запрос" |
| Релиз 2 | Разделитель, кодировка, файл без заголовка | GROUP BY, HAVING, COUNT/SUM/AVG/MIN/MAX, DISTINCT | DESCRIBE файла, справка HELP | Вывод в CSV / Markdown, сохранение в файл | Скрипт из файла (-f), Java API |
| Релиз 3 | Кеширование загруженных таблиц | INNER/LEFT JOIN, выражения и функции в SELECT | Статистика: время выполнения, число строк | Вывод в HTML | Регистрация собственных агрегатных функций |

# 4. Требования по иерархии Вигерса

Собранные требования распределены по уровням модели К. Вигерса: бизнес-требования → пользовательские требования → функциональные требования, дополненные нефункциональными требованиями (атрибутами качества и ограничениями).

## 4.1. Бизнес-требования

Бизнес-требования описывают, **зачем** создаётся продукт и какую пользу он должен принести заказчику и пользователям. Они являются верхним уровнем иерархии Вигерса: каждое требование нижних уровней должно поддерживать хотя бы одно бизнес-требование.

Таблица 6 — Бизнес-требования

| ID | Бизнес-требование |
|---|---|
| BR-1 | Сократить время подготовки типового сводного отчёта по CSV-выгрузке с десятков минут ручной работы в табличном редакторе до одной команды (не более 1 минуты, включая написание запроса знакомым с SQL пользователем). |
| BR-2 | Обеспечить воспроизводимость отчётов: один и тот же текст запроса на тех же данных всегда даёт один и тот же результат и может быть сохранён в файл-скрипт для повторного запуска. |
| BR-3 | Исключить необходимость установки и администрирования СУБД: для работы достаточно среды выполнения Java (JRE 17+) и исполняемого JAR-файла. |
| BR-4 | Дать разработчикам возможность встраивать генерацию отчётов в собственные Java-приложения через программный интерфейс (библиотеку). |
| BR-5 | Учебная цель: продемонстрировать полный цикл разработки — требования, SRS, UML, многоуровневую архитектуру интерпретатора запросов, тестирование — в рамках одного семестра. |

**Критерии успеха проекта:**

- типовые запросы из раздела «Примеры» выполняются и дают результат, совпадающий с SQLite на тех же данных;
- отчёт по файлу из 1 млн строк строится не дольше 5 секунд (см. NFR-1);
- пользователь, знающий базовый SQL, выполняет первый запрос без чтения документации, пользуясь только командой HELP.

**Границы проекта (входит в объём):**

- чтение CSV-файлов; оператор SELECT с WHERE, JOIN, GROUP BY, HAVING, ORDER BY, LIMIT;
- агрегатные функции COUNT, SUM, AVG, MIN, MAX и механизм добавления новых;
- консольный интерфейс (REPL, однократный запрос, скрипт) и Java API;
- вывод в консольную таблицу, CSV, HTML, Markdown.

**Не входит в объём (явные исключения):**

- изменение данных (INSERT, UPDATE, DELETE), создание таблиц, транзакции;
- подзапросы, оконные функции, UNION;
- графический интерфейс и построение графиков;
- сетевой режим (сервер, многопользовательский доступ).

### 4.2. Пользовательские требования

Пользовательские требования описывают, **что** пользователи должны иметь возможность делать с помощью продукта. Они записаны в формате пользовательских историй «Как <роль>, я хочу <действие>, чтобы <ценность>» с критериями приёмки.

Таблица 7 — Пользовательские требования (User Stories)

| ID | Пользовательская история | Критерий приёмки |
|---|---|---|
| US-01 | Как аналитик, я хочу выполнить SELECT-запрос к CSV-файлу, указав имя файла в FROM, чтобы получить нужные столбцы без открытия Excel. | Запрос SELECT name, grade FROM students.csv выводит два столбца и все строки файла. |
| US-02 | Как аналитик, я хочу отфильтровать строки условием WHERE (сравнения, AND/OR/NOT, LIKE, IN, BETWEEN, IS NULL), чтобы работать только с нужным подмножеством данных. | WHERE grade >= 4 AND group = 'ИТ-21' оставляет только подходящие строки. |
| US-03 | Как аналитик, я хочу сгруппировать строки (GROUP BY) и посчитать агрегаты COUNT, SUM, AVG, MIN, MAX, чтобы получать сводные показатели. | Результат совпадает с аналогичным запросом в SQLite на тех же данных. |

| ID | Пользовательская история | Критерий приёмки |
|---|---|---|
| US-04 | Как аналитик, я хочу отфильтровать группы (HAVING), отсортировать (ORDER BY) и ограничить вывод (LIMIT), чтобы строить рейтинги и топ-N. | ORDER BY avg_grade DESC LIMIT 3 выводит три строки с наибольшим средним. |
| US-05 | Как аналитик, я хочу соединить два CSV-файла (JOIN … ON), чтобы дополнять данные справочниками. | INNER и LEFT JOIN дают те же строки, что и SQLite. |
| US-06 | Как аналитик, я хочу получить понятное сообщение об ошибке с номером строки и позиции, чтобы быстро исправить опечатку в запросе. | Для SELEC name … выводится «Синтаксическая ошибка (1:1): ожидалось SELECT»; программа продолжает работу. |
| US-07 | Как аналитик, я хочу просмотреть структуру файла (DESCRIBE students.csv), чтобы узнать имена и типы столбцов перед написанием запроса. | Выводится список столбцов с выведенными типами и число строк. |
| US-08 | Как аналитик, я хочу выбрать формат вывода (таблица, CSV, HTML, Markdown) и сохранить отчёт в файл, чтобы использовать результат в письме, вики или другой программе. | Опции --format html --out report.html создают корректный HTML-файл. |
| US-09 | Как аналитик, я хочу указать разделитель, кодировку и наличие заголовка в CSV, чтобы работать с выгрузками из разных систем. | Файл в Windows-1251 с разделителем «;» читается без искажений. |
| US-10 | Как аналитик, я хочу выполнить скрипт из нескольких запросов из файла, чтобы повторять регулярные отчёты одной командой. | Команда -f weekly.sql выполняет все запросы по порядку и выводит результаты. |
| US-11 | Как разработчик-интегратор, я хочу вызвать движок из Java-кода (engine.execute(sql)) и получить объект с результатом, чтобы встраивать отчёты в своё приложение. | Метод возвращает ResultTable со схемой и строками; ошибки — проверяемыми исключениями QueryException. |
| US-12 | Как разработчик-интегратор, я хочу зарегистрировать собственную агрегатную функцию (например, MEDIAN), чтобы расширять язык без изменения исходного кода движка. | После registerFunction("median", Median::new) запрос с median(grade) выполняется. |

### 4.3. Функциональные требования

Функциональные требования определяют поведение системы, которое необходимо реализовать, чтобы пользователи могли выполнить свои задачи. Каждое требование сформулировано проверяемо и связано с пользовательскими историями (столбец «Источник»).

Таблица 8 — Функциональные требования

| ID | Функциональное требование | Источник |
|---|---|---|
| **Загрузка данных (CSV)** | | |
| FR-01 | Система должна читать CSV-файлы в соответствии с RFC 4180 (поля в кавычках, экранированные кавычки, переводы строк внутри полей). | US-01 |
| FR-02 | Система должна поддерживать задание разделителя (по умолчанию «,»; автоопределение «,» / «;» / табуляции по первой строке), кодировки (UTF-8 по умолчанию, Windows-1251) и признака наличия строки заголовка. | US-09 |
| FR-03 | Система должна автоматически определять тип столбца (INTEGER, DECIMAL, BOOLEAN, DATE в формате ISO-8601, STRING) по значениям столбца; пустое поле интерпретируется как NULL. | US-01, US-07 |
| FR-04 | Имя таблицы в FROM/JOIN задаётся путём к файлу относительно рабочего каталога; допускается псевдоним (FROM students.csv AS s). | US-01, US-05 |
| FR-05 | Повторно используемые файлы должны кешироваться в пределах сеанса; кеш сбрасывается, если файл изменился (по дате модификации). | US-10 |
| **Язык запросов** | | |
| FR-06 | Система должна разбирать оператор SELECT вида: SELECT [DISTINCT] список FROM таблица [JOIN …] [WHERE …] [GROUP BY …] [HAVING …] [ORDER BY … [ASC\|DESC]] [LIMIT n [OFFSET m]]. Грамматика приведена в разделе 3.6. | US-01…US-05 |
| FR-07 | Ключевые слова и имена функций нечувствительны к регистру; имена столбцов с пробелами задаются в двойных кавычках, строковые литералы — в одинарных. | US-01 |
| FR-08 | В выражениях поддерживаются: арифметика (+, −, *, /, %), сравнения (=, <>, <, <=, >, >=), логические AND, OR, NOT, а также LIKE, IN, BETWEEN, IS [NOT] NULL; скобки; трёхзначная логика при NULL. | US-02 |
| FR-09 | Поддерживаются агрегатные функции COUNT(*), COUNT(expr), COUNT(DISTINCT expr), SUM, AVG, MIN, MAX; агрегаты по пустой группе возвращают NULL (кроме COUNT = 0). | US-03 |
| FR-10 | Поддерживаются INNER JOIN и LEFT JOIN с условием ON (минимум — равенство столбцов). | US-05 |
| FR-11 | Поддерживаются служебные команды: DESCRIBE <файл>, HELP [тема], EXIT. | US-07 |
| **Выполнение запросов** | | |
| FR-12 | Перед выполнением система должна проверять семантику: существование файлов и столбцов, однозначность имён при JOIN, допустимость столбцов в SELECT при GROUP BY, совместимость типов в выражениях. | US-06 |

| ID | Функциональное требование | Источник |
|---|---|---|
| FR-13 | Логический порядок выполнения: FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT/OFFSET. | US-03, US-04 |
| FR-14 | Сортировка должна быть стабильной; NULL при сортировке по возрастанию располагаются в конце. | US-04 |
| FR-15 | Результат запроса представляется объектом ResultTable (схема + строки), не зависящим от формата вывода. | US-11 |
| **Формирование отчёта** | | |
| FR-16 | Система должна выводить результат в виде выровненной текстовой таблицы с заголовком и итоговой строкой «N строк, t мс»; дробные значения по умолчанию округляются до 2 знаков после запятой (настраивается опцией --precision). | US-01 |
| FR-17 | Система должна экспортировать результат в форматы CSV (RFC 4180), HTML (таблица с экранированием спецсимволов) и Markdown. | US-08 |
| FR-18 | Отчёт может быть сохранён в файл (опция --out); существующий файл перезаписывается только с флагом --force. | US-08 |
| **Интерфейсы и расширяемость** | | |
| FR-19 | Консольное приложение поддерживает три режима: интерактивный REPL (запрос завершается «;»), однократный запрос (-q) и выполнение скрипта (-f). | US-10 |
| FR-20 | Публичный Java API: класс ReportEngine с методами execute(String), describe(String), registerFunction(String, Supplier<AggregateFunction>). | US-11, US-12 |
| FR-21 | При ошибке система выводит тип ошибки, позицию (строка:столбец), фрагмент запроса с указателем «^» и не завершает сеанс REPL. | US-06 |

### 4.4. Нефункциональные требования

Нефункциональные требования задают атрибуты качества (производительность, надёжность, удобство, переносимость, сопровождаемость) и ограничения реализации. Для каждого атрибута указана измеримая характеристика, по которой требование можно проверить.

Таблица 9 — Нефункциональные требования

| ID | Нефункциональное требование |
|---|---|
| **Производительность** | |
| NFR-1 | Запрос с WHERE и GROUP BY по CSV-файлу объёмом 1 000 000 строк × 10 столбцов (~100 МБ) выполняется не более чем за 5 с на ПК с 4-ядерным процессором и SSD. |
| NFR-2 | Время разбора запроса длиной до 2 000 символов — не более 50 мс. |

| ID | Нефункциональное требование |
|---|---|
| NFR-3 | Потребление памяти при обработке файла 100 МБ — не более 1 ГБ кучи JVM (-Xmx1g). |
| **Надёжность** | |
| NFR-4 | Никакой входной запрос или некорректный CSV-файл не должен приводить к аварийному завершению с необработанным исключением; все ошибки преобразуются в QueryException с понятным сообщением. |
| NFR-5 | Исходные CSV-файлы открываются только на чтение и никогда не изменяются. |
| **Удобство использования** | |
| NFR-6 | Сообщения интерфейса и об ошибках — на русском языке; каждое сообщение об ошибке содержит позицию и, где возможно, подсказку («возможно, вы имели в виду: grade»). |
| NFR-7 | Команда HELP содержит краткую справку по синтаксису и не менее 5 примеров запросов. |
| **Переносимость** | |
| NFR-8 | Работа на Windows 10+, Linux, macOS при наличии JRE 17 или новее; поставка — один исполняемый JAR-файл. |
| NFR-9 | Корректная работа с путями и переводами строк всех трёх ОС (\n, \r\n). |
| **Сопровождаемость и расширяемость** | |
| NFR-10 | Архитектура разделена на независимые пакеты parser, ast, data, execution, output; зависимости только «сверху вниз», без циклов. |
| NFR-11 | Покрытие модульными тестами (JUnit 5) пакетов parser и execution — не менее 80 % строк. |
| NFR-12 | Добавление новой агрегатной функции или формата вывода требует создания одного класса, реализующего интерфейс, без изменения существующего кода (принцип открытости/закрытости). |
| **Ограничения реализации** | |
| NFR-13 | Язык реализации — Java 17; сборка — Gradle или Maven. |
| NFR-14 | Запрещено использовать готовые SQL-движки и парсеры (H2, SQLite JDBC, Apache Calcite, JSqlParser, ANTLR-грамматики SQL) и готовые CSV-парсеры: лексер, парсер, исполнитель и чтение CSV реализуются самостоятельно. Допустимы JUnit 5 (тесты) и picocli (разбор аргументов командной строки). |

# 5. Потребности заказчика и детали реализации

Одна из целей этапа — отделить реальные потребности заказчика от технических решений, которые заказчик иногда формулирует вместо потребности. Требования фиксируют «что» и «зачем», а «как» остаётся решением команды на этапе проектирования.

Таблица 10 — Разделение потребностей и решений

| **Высказывание заказчика** | **Реальная потребность (требование)** | **Техническая деталь / решение команды** |
|---|---|---|
| «Хочу, чтобы было как в Excel, только быстрее» | Повторяемо получать сводку по выгрузке одной командой (BR-1, BR-2) | Excel-подобный GUI — не требование; достаточно CLI с сохранением запросов в файл |
| «Сделайте на базе SQLite, там всё есть» | Работать с CSV без установки СУБД (BR-3) | Выбор хранилища — решение команды; по ограничениям задания ядро реализуем сами |
| «Нужна поддержка всего SQL» | Использовать знакомый синтаксис для типовых отчётов (US-01…US-05) | Полный стандарт не нужен: интервью показало, что используются SELECT/WHERE/GROUP BY/ORDER BY/JOIN |
| «Чтобы не падало» | Понятная диагностика ошибок с позицией (US-06, NFR-4) | Как именно реализованы восстановление после ошибок и иерархия исключений — внутреннее решение |
| «Хочу отчёт в красивом виде» | Вставлять результат в письмо/вики (US-08) | Конкретный формат определён как набор: таблица, CSV, HTML, Markdown |

## 6. Трассировка требований

Матрица трассировки показывает, что каждое бизнес-требование поддерживается пользовательскими, функциональными и нефункциональными требованиями, а «висячих» требований нет.

Таблица 11 — Матрица трассировки требований

| **Бизнес-требование** | **Пользовательские** | **Функциональные** | **Нефункциональные** |
|---|---|---|---|
| BR-1 Быстрые сводные отчёты | US-01–US-05, US-07, US-08 | FR-01–FR-17 | NFR-1, NFR-2, NFR-6, NFR-7 |
| BR-2 Воспроизводимость | US-10 | FR-05, FR-13, FR-14, FR-19 | NFR-4, NFR-5 |
| BR-3 Без установки СУБД | US-09 | FR-01–FR-03 | NFR-8, NFR-9, NFR-13 |
| BR-4 Встраивание в Java-приложения | US-11, US-12 | FR-15, FR-20 | NFR-10, NFR-12 |
| BR-5 Учебная цель | US-01–US-12 | FR-01–FR-21 | NFR-10, NFR-11, NFR-14 |

## Заключение

Выбрана тема проекта «Генератор отчётов на основе SQL-подобного языка» (SQLReport). С помощью анализа документов и аналогов, интервью, пользовательских историй и карты историй собраны и формализованы требования: 5 бизнес-требований, 12 пользовательских историй, 21 функциональное и 14 нефункциональных требований. Требования проверяемы, трассируются к бизнес-целям и разбиты на три релиза. Они служат основой для технического задания (SRS) и UML-моделирования в следующем практическом задании.